

Soft Assignment Clustering, Latent Variable Models, and the Expectation-Maximization Algorithm

A Unified Mathematical, Information-Theoretic, and Physical Foundation of Gaussian Mixture Models (GMM)

Contents

1	The Latent Variable Framework and Generative Density Estimation	2
1.1	Latent Indicator Variables and Joint Factorization	2
1.2	Posterior Responsibilities (Soft Assignments)	3
2	Maximum Likelihood Estimation and the Incomplete Data Problem	3
2.1	Pathologies of Direct Likelihood Maximization	3
3	Derivation of the Expectation-Maximization (EM) Algorithm	4
3.1	The Complete-Data Log-Likelihood	4
3.2	Decomposition of Log-Likelihood and the Evidence Lower Bound (ELBO)	4
3.3	E-Step: Expectation via KL Minimization	5
3.4	M-Step: Maximization and Analytical Updates	5
3.5	Proof of Monotonic Log-Likelihood Ascent	6
4	Information-Theoretic and Statistical Mechanics Views	6
4.1	Soft Vector Quantization and Minimum Description Length (MDL)	6
4.2	Physical Analogy: Multi-Phase Thermodynamic Ensembles	7
5	Model Selection and Covariance Regularization	7
5.1	Preventing Covariance Singularities	7
5.2	Information-Theoretic Model Selection (BIC and AIC)	7
6	Production-Grade Vectorized NumPy Implementation	8

1 The Latent Variable Framework and Generative Density Estimation

While hard-partitioning algorithms such as K -Means assign each data vector $\mathbf{x}^{(i)} \in \mathbb{R}^d$ deterministically to a single cluster, real-world data distributions frequently exhibit overlapping cluster boundaries and varying directional spreads.

****Gaussian Mixture Models (GMM)**** solve density estimation through a probabilistic, generative latent-variable framework. A GMM represents a complex, multi-modal probability density function as a convex linear combination of K continuous d -dimensional Multivariate Gaussian component densities.

Definition 1.1 (Gaussian Mixture Density). Let $\mathcal{D} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}\}$ be an empirical dataset of N i.i.d. observations sampled from an unknown data-generating distribution $p_{\text{data}}(\mathbf{x})$. A Gaussian Mixture Model parameterizes the marginal density $p(\mathbf{x} \mid \boldsymbol{\theta})$ as:

$$p(\mathbf{x} \mid \boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (1)$$

where $\boldsymbol{\theta} = \{\pi_1, \dots, \pi_K, \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K, \boldsymbol{\Sigma}_1, \dots, \boldsymbol{\Sigma}_K\}$ denotes the full parameter set, subject to:

1. **Mixing Coefficients (Priors):** $\pi_k = P(z_k = 1)$ such that $\sum_{k=1}^K \pi_k = 1$ and $\pi_k \geq 0$.
2. **Multivariate Gaussian Components:**

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)\right) \quad (2)$$

1.1 Latent Indicator Variables and Joint Factorization

To formulate GMMs as a latent variable model, we introduce an unobserved K -dimensional binary random vector $\mathbf{z} \in \{0, 1\}^K$ for each sample $\mathbf{x}$. The vector $\mathbf{z}$ follows a 1-of- K scheme (one-hot encoding) where exactly one element $z_k = 1$ and $z_j = 0$ for all $j \neq k$.

The prior probability distribution over latent vector $\mathbf{z}$ is parameterized by mixing coefficients $\boldsymbol{\pi} = [\pi_1, \dots, \pi_K]^T$:

$$p(\mathbf{z} \mid \boldsymbol{\pi}) = \prod_{k=1}^K \pi_k^{z_k} \quad (3)$$

The conditional density of observation $\mathbf{x}$ given a specific latent assignment $z_k = 1$ is:

$$p(\mathbf{x} \mid \mathbf{z}, \boldsymbol{\theta}) = \prod_{k=1}^K \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_k} \quad (4)$$

Factoring the joint density $p(\mathbf{x}, \mathbf{z} \mid \boldsymbol{\theta}) = p(\mathbf{z} \mid \boldsymbol{\pi}) p(\mathbf{x} \mid \mathbf{z}, \boldsymbol{\theta})$:

$$p(\mathbf{x}, \mathbf{z} \mid \boldsymbol{\theta}) = \prod_{k=1}^K [\pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)]^{z_k} \quad (5)$$

Summing over all K possible states of $\mathbf{z}$ recovers the marginal density $p(\mathbf{x} \mid \boldsymbol{\theta})$:

$$p(\mathbf{x} \mid \boldsymbol{\theta}) = \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z} \mid \boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (6)$$

1.2 Posterior Responsibilities (Soft Assignments)

Using Bayes' Theorem, the posterior probability that component k generated observation $\mathbf{x}^{(i)}$ —termed the **Responsibility** $\gamma_{ik} \equiv \mathbb{E}[z_{ik}]$ —is:

Definition 1.2 (Component Responsibility).

$$\gamma_{ik} = P(z_k = 1 \mid \mathbf{x}^{(i)}, \boldsymbol{\theta}) = \frac{p(z_k = 1 \mid \boldsymbol{\pi}) p(\mathbf{x}^{(i)} \mid z_k = 1, \boldsymbol{\theta})}{p(\mathbf{x}^{(i)} \mid \boldsymbol{\theta})} = \frac{\pi_k \mathcal{N}(\mathbf{x}^{(i)} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}^{(i)} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \quad (7)$$

Responsibilities form a continuous probability vector $[\gamma_{i1}, \gamma_{i2}, \dots, \gamma_{iK}]$ satisfying $\sum_{k=1}^K \gamma_{ik} = 1$, expressing the model's soft uncertainty regarding cluster origin.

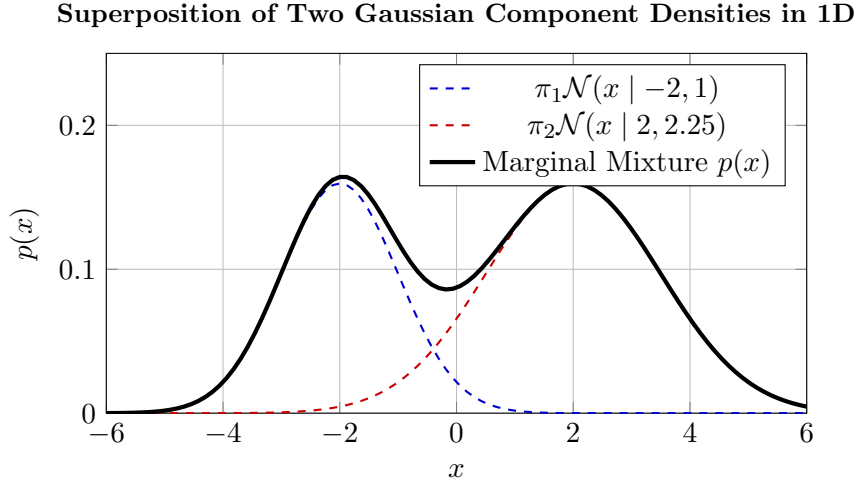


Figure 1: Generative GMM marginal probability density $p(x)$ formed by the weighted sum of two distinct 1D Gaussian components.

2 Maximum Likelihood Estimation and the Incomplete Data Problem

Under the i.i.d. assumption across N training points, the total log-likelihood function of the dataset given parameters $\boldsymbol{\theta}$ is:

$$\ln p(X \mid \boldsymbol{\theta}) = \sum_{i=1}^N \ln p(\mathbf{x}^{(i)} \mid \boldsymbol{\theta}) = \sum_{i=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}^{(i)} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right) \quad (8)$$

2.1 Pathologies of Direct Likelihood Maximization

Direct analytical optimization of $\ln p(X \mid \boldsymbol{\theta})$ by setting gradients to zero ($\nabla_{\boldsymbol{\theta}} \ln p = \mathbf{0}$) fails due to three fundamental structural barriers:

1. **Sum Inside the Logarithm:** The natural logarithm acts directly on the outer summation over K components: $\ln \left(\sum_{k=1}^K \pi_k \mathcal{N}_k \right)$. The log-arithmetic operator does not distribute across sums, coupling the parameters of all K Gaussian components together into complex non-linear equations without closed-form analytical solutions.
2. **Non-Convex Loss Landscape:** The log-likelihood surface contains multiple local maxima and saddle points caused by label switching symmetries ($K!$ identical permutations of component labels).

3. **Singularities (Infinite Likelihood Blow-Up):** If a single Gaussian component k shrinks its covariance matrix toward zero ($\Sigma_k \rightarrow 0$) while focusing its mean on a single data point ($\mu_k \rightarrow \mathbf{x}^{(j)}$), the density term evaluates to:

$$\mathcal{N}(\mathbf{x}^{(j)} \mid \mathbf{x}^{(j)}, \sigma_k^2 \mathbf{I}) = \frac{1}{(2\pi\sigma_k^2)^{d/2}} \rightarrow \infty \quad (9)$$

The log-likelihood approaches $+\infty$, creating a singular spike in parameter space that degrades unregularized optimizer runs.

3 Derivation of the Expectation-Maximization (EM) Algorithm

To bypass the non-linear coupling caused by the incomplete data log-likelihood $\ln p(X \mid \theta)$, Dempster, Laird, and Rubin (1977) formulated the **Expectation-Maximization (EM) Algorithm**.

EM optimizes the **Complete-Data Log-Likelihood** $\ln p(X, Z \mid \theta)$, where latent indicator variables $Z = \{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(N)}\}$ are assumed to be observed alongside X .

3.1 The Complete-Data Log-Likelihood

If latent vector states $\mathbf{z}^{(i)}$ were known, the complete-data log-likelihood expands cleanly:

$$\begin{aligned} \ln p(X, Z \mid \theta) &= \sum_{i=1}^N \ln p(\mathbf{x}^{(i)}, \mathbf{z}^{(i)} \mid \theta) = \sum_{i=1}^N \sum_{k=1}^K z_{ik} \ln \left[\pi_k \mathcal{N}(\mathbf{x}^{(i)} \mid \mu_k, \Sigma_k) \right] \\ &= \sum_{i=1}^N \sum_{k=1}^K z_{ik} \left[\ln \pi_k - \frac{d}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma_k| - \frac{1}{2} (\mathbf{x}^{(i)} - \mu_k)^T \Sigma_k^{-1} (\mathbf{x}^{(i)} - \mu_k) \right] \end{aligned} \quad (10)$$

Because the logarithm now acts directly on single Gaussian components (bypassing the sum over k), parameter optimization decouples into separate linear updates.

3.2 Decomposition of Log-Likelihood and the Evidence Lower Bound (ELBO)

Because latent variables Z are unobserved, we calculate the expected value of $\ln p(X, Z \mid \theta)$ under an arbitrary distribution $q(Z)$ over the latent variables.

For any valid density $q(Z)$:

$$\begin{aligned} \ln p(X \mid \theta) &= \sum_Z q(Z) \ln p(X \mid \theta) = \sum_Z q(Z) \ln \left(\frac{p(X, Z \mid \theta)}{p(Z \mid X, \theta)} \right) \\ &= \sum_Z q(Z) \ln \left(\frac{p(X, Z \mid \theta)}{q(Z)} \cdot \frac{q(Z)}{p(Z \mid X, \theta)} \right) \\ &= \sum_Z q(Z) \ln \left(\frac{p(X, Z \mid \theta)}{q(Z)} \right) + \sum_Z q(Z) \ln \left(\frac{q(Z)}{p(Z \mid X, \theta)} \right) \end{aligned} \quad (11)$$

This gives the exact fundamental decomposition:

$$\ln p(X \mid \theta) = \mathcal{F}(q, \theta) + D_{\text{KL}}(q \parallel p(Z \mid X, \theta)) \quad (12)$$

where:

- $\mathcal{F}(q, \theta) = \sum_Z q(Z) \ln \left(\frac{p(X, Z \mid \theta)}{q(Z)} \right)$ is the **Evidence Lower Bound (ELBO)**.
- $D_{\text{KL}}(q \parallel p(Z \mid X, \theta)) = -\sum_Z q(Z) \ln \left(\frac{p(Z \mid X, \theta)}{q(Z)} \right)$ is the **Kullback-Leibler (KL) Divergence** between $q(Z)$ and the true posterior $p(Z \mid X, \theta)$.

Because $D_{\text{KL}}(q \parallel p) \geq 0$ for all distributions q , the ELBO $\mathcal{F}(q, \theta)$ forms a strict lower bound on the true marginal log-likelihood:

$$\ln p(X \mid \theta) \geq \mathcal{F}(q, \theta) \quad (13)$$

3.3 E-Step: Expectation via KL Minimization

In the **E-step (Expectation Step)**, parameter vector $\boldsymbol{\theta}^{(t)}$ is held fixed. We optimize $\mathcal{F}(q, \boldsymbol{\theta}^{(t)})$ with respect to variational distribution $q(Z)$.

Because $\ln p(X | \boldsymbol{\theta}^{(t)})$ is independent of $q(Z)$, maximizing $\mathcal{F}(q, \boldsymbol{\theta}^{(t)})$ is equivalent to minimizing $D_{\text{KL}}(q \| p(Z | X, \boldsymbol{\theta}^{(t)}))$.

Setting $q(Z)$ equal to the exact posterior distribution under current parameters $\boldsymbol{\theta}^{(t)}$:

$$q^*(Z) = p(Z | X, \boldsymbol{\theta}^{(t)}) \quad (14)$$

This reduces $D_{\text{KL}}(q^* \| p) = 0$, making the lower bound tight at the current parameter point:

$$\mathcal{F}(q^*, \boldsymbol{\theta}^{(t)}) = \ln p(X | \boldsymbol{\theta}^{(t)}) \quad (15)$$

Evaluating the expected complete-data log-likelihood $\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$:

$$\begin{aligned} \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) &\equiv \mathbb{E}_{Z|X, \boldsymbol{\theta}^{(t)}} [\ln p(X, Z | \boldsymbol{\theta})] = \sum_Z p(Z | X, \boldsymbol{\theta}^{(t)}) \ln p(X, Z | \boldsymbol{\theta}) \\ &= \sum_{i=1}^N \sum_{k=1}^K \gamma_{ik}^{(t)} \left[\ln \pi_k + \ln \mathcal{N}(\mathbf{x}^{(i)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right] \end{aligned} \quad (16)$$

where $\gamma_{ik}^{(t)} = \mathbb{E}[z_{ik}] = \frac{\pi_k^{(t)} \mathcal{N}(\mathbf{x}^{(i)} | \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}{\sum_{j=1}^K \pi_j^{(t)} \mathcal{N}(\mathbf{x}^{(i)} | \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}$.

3.4 M-Step: Maximization and Analytical Updates

In the **M-step (Maximization Step)**, distribution $q(Z)$ is held fixed at responsibilities $\gamma_{ik}^{(t)}$, and we update parameters by maximizing $\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)})$:

$$\boldsymbol{\theta}^{(t+1)} = \arg \max_{\boldsymbol{\theta}} \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}) \quad (17)$$

Setting partial derivatives with respect to each component parameter to zero yields the updated equations:

1. Updating Component Means $\boldsymbol{\mu}_k$

$$\frac{\partial \mathcal{Q}}{\partial \boldsymbol{\mu}_k} = \sum_{i=1}^N \gamma_{ik}^{(t)} \frac{\partial}{\partial \boldsymbol{\mu}_k} \left(-\frac{1}{2} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k) \right) = \mathbf{0} \quad (18)$$

$$\sum_{i=1}^N \gamma_{ik}^{(t)} \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k) = \mathbf{0} \implies \sum_{i=1}^N \gamma_{ik}^{(t)} \mathbf{x}^{(i)} = \boldsymbol{\mu}_k \sum_{i=1}^N \gamma_{ik}^{(t)} \quad (19)$$

Defining $N_k = \sum_{i=1}^N \gamma_{ik}^{(t)}$ as the effective sample count assigned to component k :

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik}^{(t)} \mathbf{x}^{(i)} \quad (20)$$

2. Updating Component Covariances $\boldsymbol{\Sigma}_k$

Taking the derivative of $\mathcal{Q}$ with respect to precision matrix $\boldsymbol{\Sigma}_k^{-1}$:

$$\frac{\partial \mathcal{Q}}{\partial \boldsymbol{\Sigma}_k^{-1}} = \sum_{i=1}^N \gamma_{ik}^{(t)} \left(\frac{1}{2} \boldsymbol{\Sigma}_k - \frac{1}{2} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k^{(t+1)}) (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k^{(t+1)})^T \right) = \mathbf{0} \quad (21)$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik}^{(t)} \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k^{(t+1)} \right) \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k^{(t+1)} \right)^T \quad (22)$$

3. Updating Mixing Coefficients π_k

To enforce constraint $\sum_{k=1}^K \pi_k = 1$, we introduce Lagrange multiplier λ :

$$\mathcal{L}_{\pi}(\pi, \lambda) = \sum_{i=1}^N \sum_{k=1}^K \gamma_{ik}^{(t)} \ln \pi_k + \lambda \left(1 - \sum_{k=1}^K \pi_k \right) \quad (23)$$

Taking partial derivative with respect to π_k :

$$\frac{\partial \mathcal{L}_{\pi}}{\partial \pi_k} = \sum_{i=1}^N \frac{\gamma_{ik}^{(t)}}{\pi_k} - \lambda = 0 \implies N_k = \lambda \pi_k \quad (24)$$

Summing over all K : $\sum_k N_k = \lambda \sum_k \pi_k \implies N = \lambda$. Substituting $\lambda = N$:

$$\pi_k^{(t+1)} = \frac{N_k}{N} \quad (25)$$

3.5 Proof of Monotonic Log-Likelihood Ascent

Theorem 3.1 (Monotonic Ascent Theorem). Every EM iteration guarantees a non-decreasing step in the incomplete-data log-likelihood:

$$\ln p(X | \theta^{(t+1)}) \geq \ln p(X | \theta^{(t)}) \quad (26)$$

Proof. By definition of the ELBO decomposition:

$$\ln p(X | \theta^{(t)}) = \mathcal{F}(q^{(t)}, \theta^{(t)}) + \underbrace{D_{\text{KL}}(q^{(t)} \parallel p(Z | X, \theta^{(t)}))}_{0 \text{ after E-step}} = \mathcal{F}(q^{(t)}, \theta^{(t)}) \quad (27)$$

In the M-step, $\theta^{(t+1)}$ maximizes $\mathcal{F}(q^{(t)}, \theta)$ with respect to θ :

$$\mathcal{F}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \theta^{(t)}) \quad (28)$$

Because KL divergence is non-negative ($D_{\text{KL}} \geq 0$):

$$\begin{aligned} \ln p(X | \theta^{(t+1)}) &= \mathcal{F}(q^{(t)}, \theta^{(t+1)}) + D_{\text{KL}}(q^{(t)} \parallel p(Z | X, \theta^{(t+1)})) \\ &\geq \mathcal{F}(q^{(t)}, \theta^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \theta^{(t)}) = \ln p(X | \theta^{(t)}) \end{aligned} \quad (29)$$

Thus, the likelihood increases monotonically at every iteration. $\square$

4 Information-Theoretic and Statistical Mechanics Views

4.1 Soft Vector Quantization and Minimum Description Length (MDL)

In rate-distortion theory, GMM acts as a **Soft Vector Quantizer**.

Instead of mapping a continuous signal $\mathbf{x}$ to a single discrete codebook vector with zero margin, GMM transmits a soft posterior distribution $\gamma_i = [\gamma_{i1}, \dots, \gamma_{iK}]^T$. The expected description length required to code the signal under Codebook θ equals the cross-entropy $H(p_{\text{data}}, p_{\text{model}})$. Minimizing cross-entropy is equivalent to maximizing the likelihood under GMM density estimation.

4.2 Physical Analogy: Multi-Phase Thermodynamic Ensembles

GMM mathematics maps onto multi-phase thermodynamic fluid systems under thermal equilibrium.

Consider a physical fluid mixture comprising K distinct fluid phases coexisting at thermal equilibrium.

- The mixing coefficients π_k represent **Phase Volume Fractions** of the fluid states.
- The exponent in the Gaussian component density functions corresponds to a d -dimensional **Harmonic Energy Potential** $E_k(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)$.
- Responsibilities γ_{ik} act as **Boltzmann Thermal State Probabilities** at inverse temperature $\beta = 1$:

$$\gamma_{ik} = \frac{e^{-E_k(\mathbf{x}^{(i)})}}{\sum_{j=1}^K e^{-E_j(\mathbf{x}^{(i)})}} = \frac{e^{-E_k(\mathbf{x}^{(i)})}}{Z(\mathbf{x}^{(i)})} \quad (30)$$

where $Z(\mathbf{x}^{(i)})$ is the local **Partition Function**.

As component covariance matrices vanish ($\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I} \rightarrow 0$), the physical temperature vanishes ($T \rightarrow 0^+$), collapsing soft Boltzmann responsibilities into hard K -Means Voronoi step boundaries.

5 Model Selection and Covariance Regularization

5.1 Preventing Covariance Singularities

To prevent likelihood blow-up caused by component collapse ($\boldsymbol{\Sigma}_k \rightarrow 0$), numerical implementations add a small positive diagonal ridge (shrinkage) $\epsilon \mathbf{I}_d$ ($\epsilon \approx 10^{-6}$) to every component covariance matrix during the M-step:

$$\boldsymbol{\Sigma}_{k,\text{reg}}^{(t+1)} = \boldsymbol{\Sigma}_k^{(t+1)} + \epsilon \mathbf{I}_d \quad (31)$$

This guarantees that $\boldsymbol{\Sigma}_k$ remains strictly positive-definite ($|\boldsymbol{\Sigma}_k| > 0$) and invertible across all iterations.

5.2 Information-Theoretic Model Selection (BIC and AIC)

Increasing component count K monotonically increases training log-likelihood $\ln p(X | \hat{\boldsymbol{\theta}}_K)$, risking overfitting. To select optimal component count K^* , we penalize model complexity using information criteria:

Definition 5.1 (Bayesian Information Criterion (BIC)).

$$\text{BIC}(K) = p \ln(N) - 2 \ln p(X | \hat{\boldsymbol{\theta}}_K) \quad (32)$$

Definition 5.2 (Akaike Information Criterion (AIC)).

$$\text{AIC}(K) = 2p - 2 \ln p(X | \hat{\boldsymbol{\theta}}_K) \quad (33)$$

where p is the total number of free trainable parameters in the model.

For full covariance matrices in d dimensions:

$$p = \underbrace{(K-1)}_{\text{priors } \pi_k} + \underbrace{K \cdot d}_{\text{means } \boldsymbol{\mu}_k} + \underbrace{K \cdot \frac{d(d+1)}{2}}_{\text{covariances } \boldsymbol{\Sigma}_k} \quad (34)$$

Optimal K^* is selected by minimizing BIC or AIC.

6 Production-Grade Vectorized NumPy Implementation

The following Python class implements a complete Gaussian Mixture Model using vectorized NumPy operations, including the ****Log-Sum-Exp Trick**** for numerical stability, covariance regularization, BIC/AIC scoring, and full EM optimization.

```
1 import numpy as np
2
3 class UniversalGaussianMixtureEngine:
4     """
5     Production-grade Gaussian Mixture Model (GMM) Engine using Expectation-
6     Maximization (EM).
7     Features Log-Sum-Exp stability, Cholesky-based density evaluations,
8     covariance regularization, and Model Selection Criteria (BIC/AIC).
9     """
10    def __init__(self, n_components: int = 3, max_iter: int = 100,
11                 tol: float = 1e-3, reg_covar: float = 1e-6):
12        self.n_components = n_components
13        self.max_iter = max_iter
14        self.tol = tol
15        self.reg_covar = reg_covar
16
17        self.weights_ = None          # Mixing coefficients pi_k (K,)
18        self.means_ = None            # Component means mu_k (K, d)
19        self.covariances_ = None      # Component covariances Sigma_k (K, d, d)
20
21    )
22
23    self.converged_ = False
24    self.lower_bound_ = -np.inf # Log-likelihood track
25
26    def _estimate_log_gaussian_densities(self, X: np.ndarray) -> np.ndarray
27    :
28        """
29        Computes log N(X | mu_k, Sigma_k) matrix (N, K) using Cholesky
30        factor
31        for high numerical stability.
32        """
33        N, d = X.shape
34        K = self.n_components
35        log_densities = np.zeros((N, K))
36
37        for k in range(K):
38            mu_k = self.means_[k]
39            Sigma_k = self.covariances_[k] + self.reg_covar * np.eye(d)
40
41            try:
42                # Cholesky decomposition: Sigma = L * L^T
43                L = np.linalg.cholesky(Sigma_k)
44            except np.linalg.LinAlgError:
45                Sigma_k += 1e-4 * np.eye(d)
46                L = np.linalg.cholesky(Sigma_k)
47
48            # Solve triangular system: L * V = (X - mu)^T => V = L^-1 * (X
49            - mu)^T
50            X_centered = X - mu_k
51            V = np.linalg.solve_triangular(L, X_centered.T, lower=True)
52
53            # Squared Mahalanobis distance: D_M^2 = sum(V^2, axis=0)
54            mahalanobis_sq = np.sum(V**2, axis=0)
55
56            # Log determinant: log|Sigma_k| = 2 * sum(log(diag(L)))
```

```

50         log_det = 2.0 * np.sum(np.log(np.diagonal(L)))
51
52         log_densities[:, k] = -0.5 * (d * np.log(2.0 * np.pi) + log_det
+ mahalanobis_sq)
53
54         return log_densities
55
56     def _e_step(self, X: np.ndarray):
57         """
58         Expectation Step: Computes log responsibilities gamma_ik using
59         the Log-Sum-Exp trick for numerical stability.
60         """
61         log_gaussian = self._estimate_log_gaussian_densities(X) # (N, K)
62         log_weighted = log_gaussian + np.log(self.weights_ + 1e-12) # (N, K)
63     )
64
65     # Log-Sum-Exp Trick: log(sum(exp(Z))) = max(Z) + log(sum(exp(Z -
max(Z))))
66     max_log = np.max(log_weighted, axis=1, keepdims=True)
67     log_likelihood_per_sample = max_log + np.log(np.sum(np.exp(
log_weighted - max_log), axis=1, keepdims=True))
68
69     log_resp = log_weighted - log_likelihood_per_sample # Normalized
log responsibilities
70     responsibilities = np.exp(log_resp)
71
72     total_log_likelihood = np.sum(log_likelihood_per_sample)
73     return responsibilities, total_log_likelihood
74
75     def _m_step(self, X: np.ndarray, responsibilities: np.ndarray):
76         """Maximization Step: Updates pi_k, mu_k, and Sigma_k given
responsibilities."""
77         N, d = X.shape
78         K = self.n_components
79
80         # Effective number of points per component: N_k = sum_i(gamma_ik)
N_k = np.sum(responsibilities, axis=0) + 1e-10 # (K,)
81
82         # 1. Update Mixing Coefficients: pi_k = N_k / N
self.weights_ = N_k / N
83
84         # 2. Update Means: mu_k = (1 / N_k) * sum_i(gamma_ik * x_i)
self.means_ = (responsibilities.T @ X) / N_k[:, np.newaxis] # (K, d)
85
86     )
87
88     # 3. Update Covariances: Sigma_k = (1 / N_k) * sum_i(gamma_ik * (
x_i - mu_k)(x_i - mu_k)^T)
89     for k in range(K):
90         X_centered = X - self.means_[k] # (N, d)
91         weighted_diff = X_centered * responsibilities[:, k:k+1] # (N, d)
92     )
93
94     Sigma_k = (weighted_diff.T @ X_centered) / N_k[k]
95     # Add shrinkage regularization
self.covariances_[k] = Sigma_k + self.reg_covar * np.eye(d)
96
97     def fit(self, X: np.ndarray):
98         """Fits GMM parameters via Expectation-Maximization (EM)."""
99         X = np.asarray(X, dtype=np.float64)
100         N, d = X.shape
K = self.n_components

```

```

101
102     # Initialize parameters using K-Means-like uniform sampling
103     random_indices = np.random.choice(N, K, replace=False)
104     self.means_ = X[random_indices].copy()
105     self.weights_ = np.full(K, 1.0 / K)
106
107     # Initialize covariances as empirical dataset variance
108     global_var = np.var(X, axis=0) + self.reg_covar
109     self.covariances_ = np.array([np.diag(global_var) for _ in range(K)
])
110
111     prev_log_likelihood = -np.inf
112
113     for loop_iter in range(self.max_iter):
114         # E-Step
115         responsibilities, log_likelihood = self._e_step(X)
116
117         # Check convergence
118         if np.abs(log_likelihood - prev_log_likelihood) < self.tol:
119             self.converged_ = True
120             break
121
122         prev_log_likelihood = log_likelihood
123
124         # M-Step
125         self._m_step(X, responsibilities)
126
127     self.lower_bound_ = log_likelihood
128     return self
129
130     def predict_proba(self, X: np.ndarray) -> np.ndarray:
131         """Evaluates soft posterior responsibilities gamma_ik for test
samples."""
132         X = np.asarray(X, dtype=np.float64)
133         responsibilities, _ = self._e_step(X)
134         return responsibilities
135
136     def predict(self, X: np.ndarray) -> np.ndarray:
137         """Assigns samples to component with maximum posterior
responsibility."""
138         probs = self.predict_proba(X)
139         return np.argmax(probs, axis=1)
140
141     def score_samples(self, X: np.ndarray) -> np.ndarray:
142         """Returns log-likelihood per sample: log p(x)."""
143         X = np.asarray(X, dtype=np.float64)
144         log_gaussian = self._estimate_log_gaussian_densities(X)
145         log_weighted = log_gaussian + np.log(self.weights_ + 1e-12)
146         max_log = np.max(log_weighted, axis=1, keepdims=True)
147         return (max_log + np.log(np.sum(np.exp(log_weighted - max_log),
axis=1, keepdims=True))).ravel()
148
149     def bic(self, X: np.ndarray) -> float:
150         """Calculates Bayesian Information Criterion (BIC)."""
151         N, d = X.shape
152         K = self.n_components
153         # Number of free parameters: (K-1) priors + K*d means + K*d*(d+1)/2
covariances
154         num_params = (K - 1) + K * d + K * d * (d + 1) // 2
155         return num_params * np.log(N) - 2.0 * np.sum(self.score_samples(X))

```

```

156
157     def aic(self, X: np.ndarray) -> float:
158         """Calculates Akaike Information Criterion (AIC)."""
159         N, d = X.shape
160         K = self.n_components
161         num_params = (K - 1) + K * d + K * d * (d + 1) // 2
162         return 2.0 * num_params - 2.0 * np.sum(self.score_samples(X))
163
164
165 if __name__ == "__main__":
166     np.random.seed(42)
167
168     # Generate Synthetic 3-Component Gaussian Mixture Dataset
169     N_per_comp = 200
170     comp1 = np.random.multivariate_normal([-3.0, -2.0], [[1.0, 0.6], [0.6,
171     1.0]], N_per_comp)
172     comp2 = np.random.multivariate_normal([3.0, 3.0], [[1.5, -0.4], [-0.4,
173     0.8]], N_per_comp)
174     comp3 = np.random.multivariate_normal([0.0, 4.0], [[0.5, 0.0], [0.0,
175     0.5]], N_per_comp)
176
177     X_data = np.vstack([comp1, comp2, comp3])
178
179     # Fit Gaussian Mixture Model
180     gmm = UniversalGaussianMixtureEngine(n_components=3, max_iter=200, tol
181     =1e-4)
182     gmm.fit(X_data)
183
184     soft_probs = gmm.predict_proba(X_data[:5])
185     hard_labels = gmm.predict(X_data)
186
187     print("=== GAUSSIAN MIXTURE MODEL (GMM) EM INITIALIZATION SUCCESSFUL
188     ===")
189     print(f"Convergence Achieved      : {gmm.converged_}")
190     print(f"Total Log-Likelihood      : {gmm.lower_bound_:.4f}")
191     print(f"Estimated Model BIC Score : {gmm.bic(X_data):.2f}")
192     print(f"Estimated Model AIC Score : {gmm.aic(X_data):.2f}")
193     print(f"Learned Mixing Weights    : {gmm.weights_}")
194     print(f"Learned Component Means:\n{gmm.means_}")

```

Listing 1: Vectorized NumPy implementation of Gaussian Mixture Models (GMM) with EM.